Chapter 8

Convolutional neural networks

Chapter based on notes by Dr Kevin Webster

A convolutional neural network (CNN) is a type of neural network with a special structure. It can be seen as a special case of the multilayer perceptron architecture that builds certain assumptions into the design of the model, in particular using **local connectivity** and **equivariance**.

An important motivating application for CNNs is computer vision, as the architectural design of these networks mimics the visual system, where neurons respond to stimulus in a restricted region of the visual field (Hubel 1959 [3]). This concept led initially to the development of the neocognitron (Fukushima 1980 [2]), and later to the modern convolutional neural network trained by backpropagation (LeCun et al 1989 [4]).

In this exposition, **we will focus on CNNs for image processing**, using 2-D convolutions. However, convolutional neural networks have also been very successful when applied to time series data, using 1-D convolutions. They can also be applied to 3-D image processing tasks, or video analysis, with 3-D convolutions.

1. The convolution operation

The convolution operation for two (Lebesgue integrable) functions h and k is defined as:

$$(h * k)(t) = \int_{-\infty}^{\infty} h(\tau)k(t-\tau)d\tau.$$

It can be described as the weighted average of the function h according to the weighting function (or **kernel**) k at each point in time t. As t changes, the weighting function emphasises different parts of the input function h.

In practice (and in the context of CNNs), one has discrete data, thus we need to work instead with discrete convolutions:

$$(h * k)(t) = \sum_{\tau=-\infty}^{\infty} h(\tau)k(t-\tau).$$

In this case, we assume that both h and k (and the convolution $(h * k)$) take integer arguments. In addition, when the kernel function k has finite support, we can write the above as a finite summation.

CNNs with image inputs use 2-D discrete convolutions, which are defined as:

$$(\mathbf{h} * \mathbf{k})(i,j) = \sum_{m,n} h(m,n)k(i-m, j-n).$$

In practice, in CNNs one implements the **cross-correlation** operation, which is the same as above but changing the orientation of the arguments:

$$(\mathbf{h} * \mathbf{k})(i,j) = \sum_{m,n} h(i+m, j+n)k(m,n). \tag{8.1}$$

and we refer to this operation in CNNs as a *convolution*. In the above, $h(i,j)$ and $k(i,j)$ denote the (i,j)-th elements of the matrices $\mathbf{h} \in \mathbb{R}^{n_h \times n_w}$ and $\mathbf{k} \in \mathbb{R}^{k_h \times k_w}$ respectively, where n_h and n_w are the image height and width in pixels, and k_h and k_w are the kernel height and width in pixels (typically much smaller than the ones of the image). In the convolution operation (8.1), the kernel $\mathbf{k}$ is progressively swept through the input $\mathbf{h}$, and different weights of the kernel $\mathbf{k}$ will pick out different features in the input $\mathbf{h}^{(k-1)}$.

As an example, consider a grayscale image $\mathbf{x} =: \mathbf{h}^{(0)} \in \mathbb{R}^{7\times 7}$ with height and width $n_h = n_w = 7$, as illustrated in the following Figure 8.1.

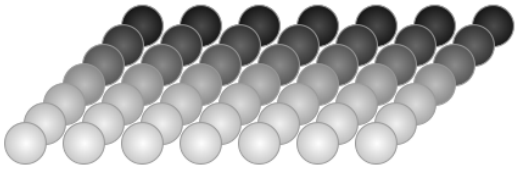


Figure 8.1. Pixels in a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times 7}$.

Suppose we define a kernel matrix $\mathbf{k} \in \mathbb{R}^{3\times 4}$. The operation 8.1 can be visualised as in Figure 8.2 as sweeping the convolutional kernel $\mathbf{k}$ across the input image. At each step, the weights of the kernel matrix $\mathbf{k} \in \mathbb{R}^{3\times 4}$ are multiplied pointwise by the values of the pixels outlined in red and summed together to produce the pre-activation of the neuron in the next hidden layer.

Key message: In CNNs, a **convolutional layer** consists of the convolution operation 8.1 plus a bias term, followed by a pointwise activation function:

$$\mathbf{h}^{(l)} = \sigma\left((\mathbf{h}^{(l-1)} * \mathbf{k}^{(l-1)}) + b^{(l-1)}\right). \tag{8.2}$$

Here the superscript (l) indexes the layer, as in Chapter 7, and the bias $b^{(l-1)} \in \mathbb{R}$ is added pointwise to the output of the convolution operation $(\mathbf{h}^{(l-1)} * \mathbf{k}^{(l-1)}) \in \mathbb{R}^{(n_h-k_h+1)\times(n_w-k_w+1)}$. The output of 8.2 is sometimes referred to as a **feature map**, and the kernel $\mathbf{k}$ is also referred to as a **filter**. Each filter $\mathbf{k}$ in the network consists of a matrix of learnable weights, which are trained via the minimisation of an appropriately defined loss function by error backpropagation (see Chapter 7).

Note that **the convolution operation introduces a translational equivariance property** in convolutional layers. That is, if the input image is translated, then the activations in the next hidden layer are also translated accordingly. Another way to view



Figure 8.2. 2-D convolution with kernel $\mathbf{k} \in \mathbb{R}^{3\times 4}$ operating on a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times 7}$

this is that the convolutional kernel searches for the same features across the input image, thanks to *weight sharing*, i.e., the use of the same small filters across the input image.

Weight sharing is the key of the success of CNNs in pattern recognition tasks for computer vision: it enables detection of localised features and the equivariance of predictions, which ensures that feature detection is robust to its exact position.

2. Multi-channel inputs and outputs

The operations 8.1 and 8.2 can easily be extended to inputs with multiple channels. Consider, for example, an input RGB image, which has three channel values per pixel (Figure 8.3).

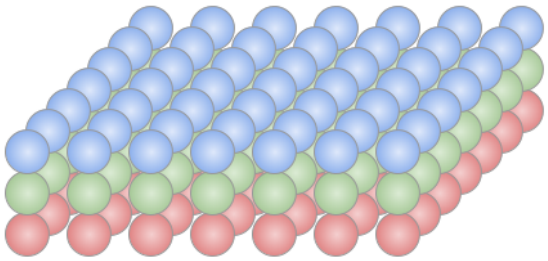


Figure 8.3. Pixels in an RGB input image $\mathbf{x} \in \mathbb{R}^{7\times 7\times 3}$.

The input is now a rank-3 tensor $\mathbf{x} = \mathbf{h}^{(0)} \in \mathbb{R}^{7\times 7\times 3}$, and correspondingly we require a rank-3 kernel tensor $\mathbf{k} \in \mathbb{R}^{k_h\times k_w\times 3}$. For illustration we will again choose $k_h = 3$, $k_w = 4$. The operation 8.1 now becomes:

$$(\mathbf{h} * \mathbf{k})(i,j) = \sum_{m,n,p} h(i+m, j+n, p)k(m,n,p).$$

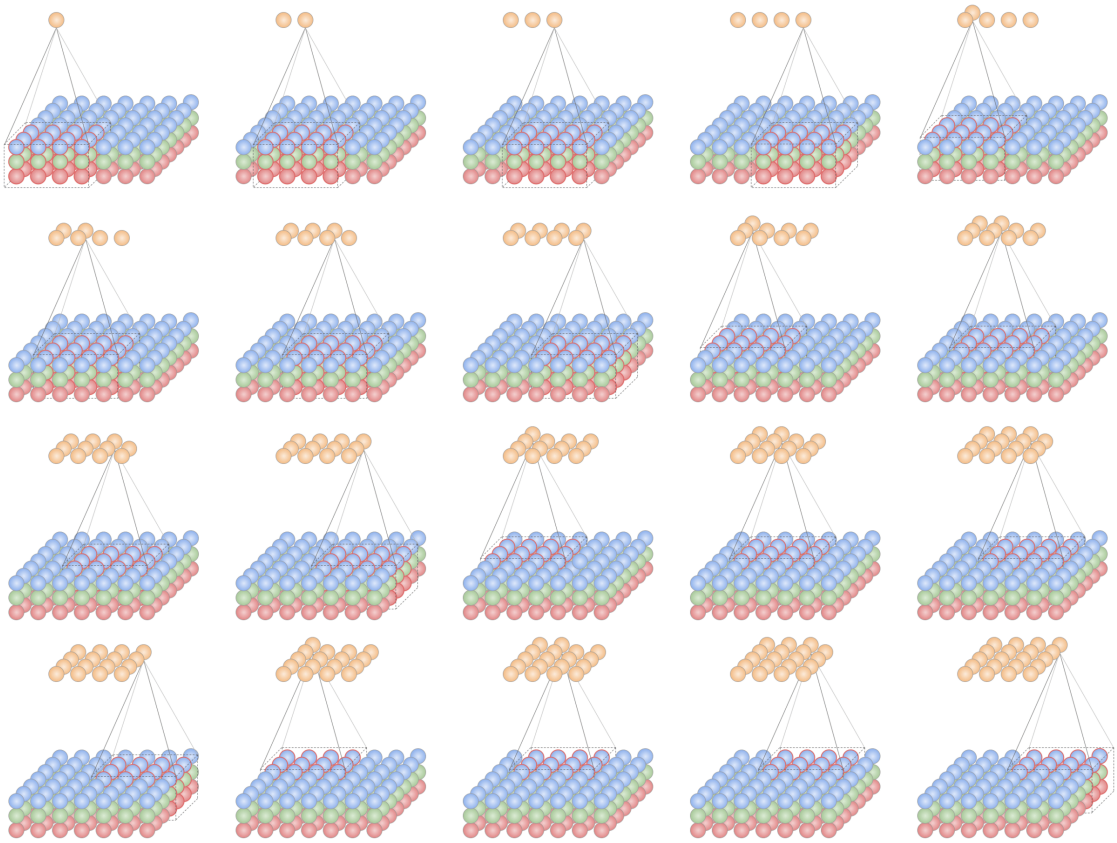


Figure 8.4. 2-D convolution with kernel $\mathbf{k} \in \mathbb{R}^{3\times4\times3}$ operating on an RGB input image $\mathbf{x} \in \mathbb{R}^{7\times7\times3}$.

This is visualised in Figure 8.4, where the kernel $\mathbf{k}$ again sweeps over the input image, this time multiplying a $3 \times 4 \times 3$ block of input pixels (outlined in red) elementwise by the values of the kernel tensor $\mathbf{k}$, and summing the results to produce the output neuron pre-activation.

In convolutional layers, **many filters are stacked on top of each other, so as to produce a multichannel output**. In practice, we implement this with a rank-4 kernel tensor $\mathbf{k} \in \mathbb{R}^{k_h\times k_w\times c_{in}\times c_{out}}$, where c_{in} are the number of channels in the input, and c_{out} are the number of channels in the output:

$$(\mathbf{h} * \mathbf{k})(i, j, q) = \sum_{m,n,p} h(i + m, j + n, p)k(m, n, p, q).$$

This is visualised in Figure 8.5, for an input $\mathbf{x} \in \mathbb{R}^{7\times7\times3}$ and kernel tensor $\mathbf{k} \in \mathbb{R}^{3\times4\times3\times2}$.

The convolutional layer operation 8.2 now becomes the following

$$\mathbf{h}^{(l)} = \sigma\left((\mathbf{h}^{(l-1)} * \mathbf{k}^{(l-1)}) + \mathbf{b}^{(l-1)}\right),$$

where now the bias $\mathbf{b}^{(l-1)} \in \mathbb{R}^{c_{out}}$ is added pixel-wise to the output of the convolution operation:

$$(\mathbf{h}^{(l-1)} * \mathbf{k}^{(l-1)}) \in \mathbb{R}^{(n_h-k_h+1)\times(n_w-k_w+1)\times c_{out}}.$$

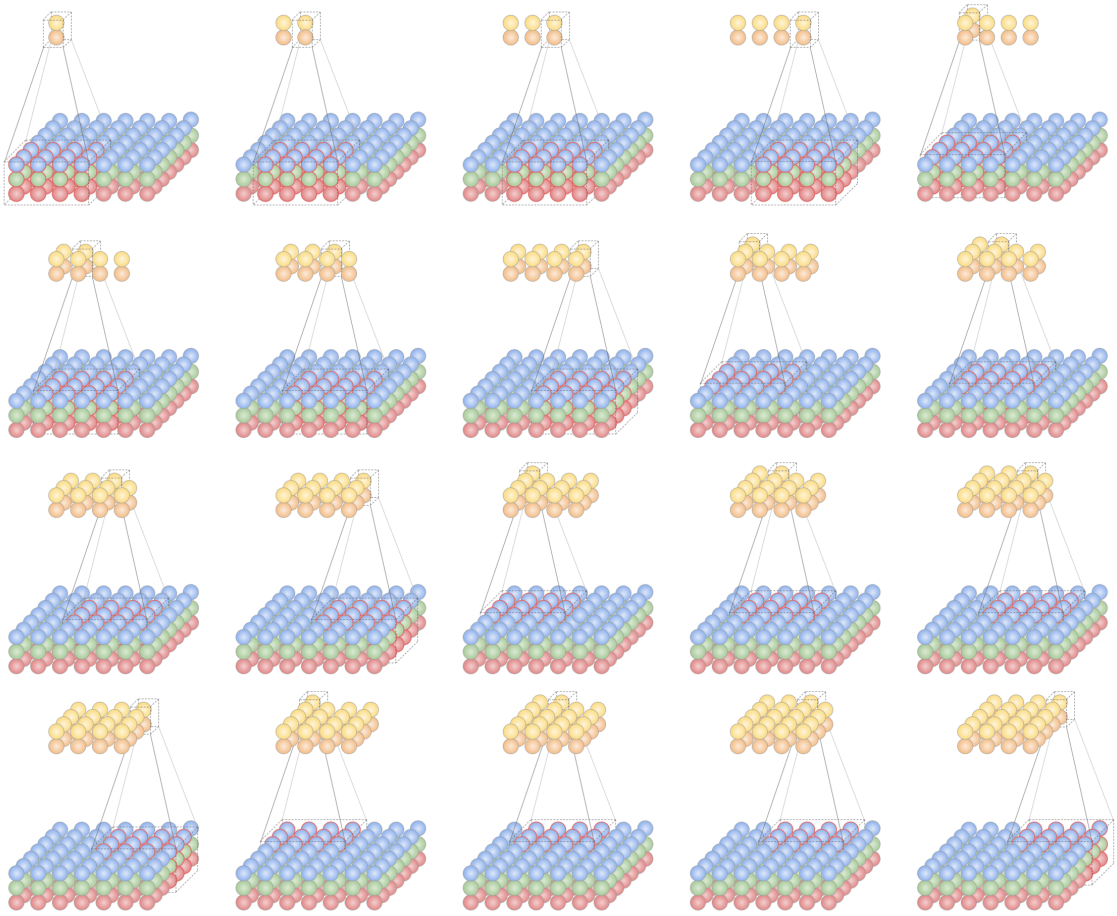


Figure 8.5. 2-D convolution with kernel $\mathbf{k} \in \mathbb{R}^{3\times4\times3}$ operating on an RGB input image $\mathbf{x} \in \mathbb{R}^{7\times7\times3\times2}$ with 2 filters

3. Pooling layers

In many CNN models, **convolutional layers are alternated with pooling layers that downsample the spatial dimensions of a layer**, by computing a summary statistic of (often non-overlapping) regions of the input layer's post-activations.

A typical pooling layer type is the **max pooling** layer (Zhou & Chellappa 1988 [6]) which takes the maximum activation in a region as the single neuron activation for that region. For example, we could divide the input layer into 2×2 squares and take the maximum value for each square. This results in halving the spatial dimensions of the following layer.

Pooling operations are usually performed separately for each input channel, so that the spatial dimensions are downsampled, but the channel dimensions stay the same. Other common pooling operations include average pooling, or computing the L_2 norm of the pixel values within each input region.

4. Padding and strides

Padding and strides are additional properties of convolutional (and pooling) layers that can give some flexibility over the spatial dimensions of the output. In this section, we will define these properties and the effects they have on the input and output dimensions of a

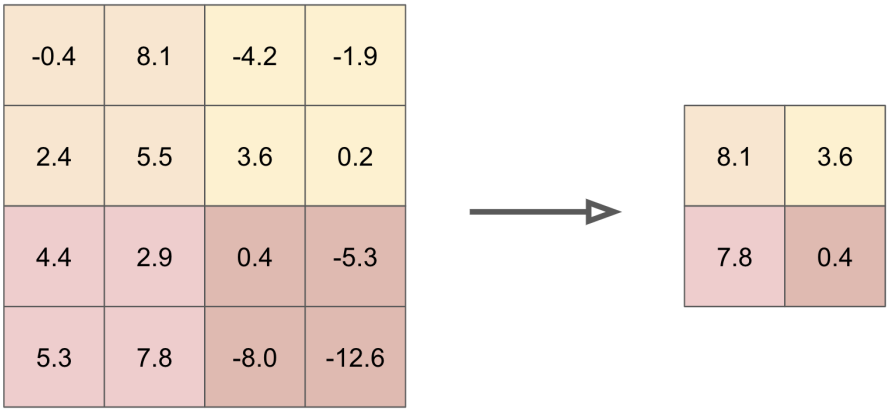


Figure 8.6. Max pooling using 2×2 pooling windows.

convolutional layer. For a more complete guide to convolutional arithmetic, see Dumoulin & Visin 2016 [1].

4.1. Padding. Recall that in our earlier example, an input grayscale image $\mathbf{x} \in \mathbb{R}^{7\times7\times1}$ convolved with a kernel $\mathbf{k} \in \mathbb{R}^{3\times4\times1\times1}$ produced an output of size $\mathbf{h} \in \mathbb{R}^{5\times4\times1}$, visualised in Figure 8.2. In general, for a spatial dimension of size i and a kernel of width k, the output size o is given by:

$$o = i - k + 1 \tag{8.3}$$

In many model architectures, it is desirable to keep the spatial dimensions the same in the output of a convolutional layer. This can be achieved by padding the input layer with zeros.

In the case of our kernel tensor $\mathbf{k} \in \mathbb{R}^{3\times4\times1\times1}$, if we add 2 zeros in the first dimension and 3 zeros in the second dimension (distributed on either side of the image), this will result in an output $\mathbf{h} \in \mathbb{R}^{7\times7\times1}$ that has the same spatial dimensions as the input $\mathbf{x}$. This type of padding is known as "SAME" padding - see Figure 8.7 for a complete visualisation of this.

If p zeros are added to our input size i with kernel width k, then the output size o is given by:

$$o = i + p - k + 1 \tag{8.4}$$

If $p = k - 1$ ("SAME" padding) then $o = i$. If $p = 0$ ("VALID" padding) then we recover 8.3 and $o = i - k + 1$. There is also "FULL" padding where $p = 2(k - 1)$, so that $o = i + k - 1$, although this is less common.

4.2. Strides. Convolutions may also use a stride s, which is the distance between consecutive positions of the kernel. So far, all of our examples have used $s = 1$, however it is easy to see that using $s > 1$ leads to a downsampling of the input. Figure 8.8 shows our input $\mathbf{x} \in \mathbb{R}^{7\times7\times1}$, this time with a kernel $\mathbf{k} \in \mathbb{R}^{3\times3\times1\times1}$, "SAME" padding, and a stride $s = 2$ in both spatial dimensions.

In the above example, the stride and kernel size exactly divides the input and padding size, however this does not need to be the case, such as in Figure 8.9 where $\mathbf{x} \in \mathbb{R}^{7\times6\times1}$.

In this example the input $\mathbf{x} \in \mathbb{R}^{7\times6\times1}$ has been downsampled to an output $\mathbf{h} \in \mathbb{R}^{4\times3\times1}$. Note that if the input image size was instead $7 \times 5 \times 1$ then the output size would again be $4 \times 3 \times 1$ as can be seen in Figure 8.10.

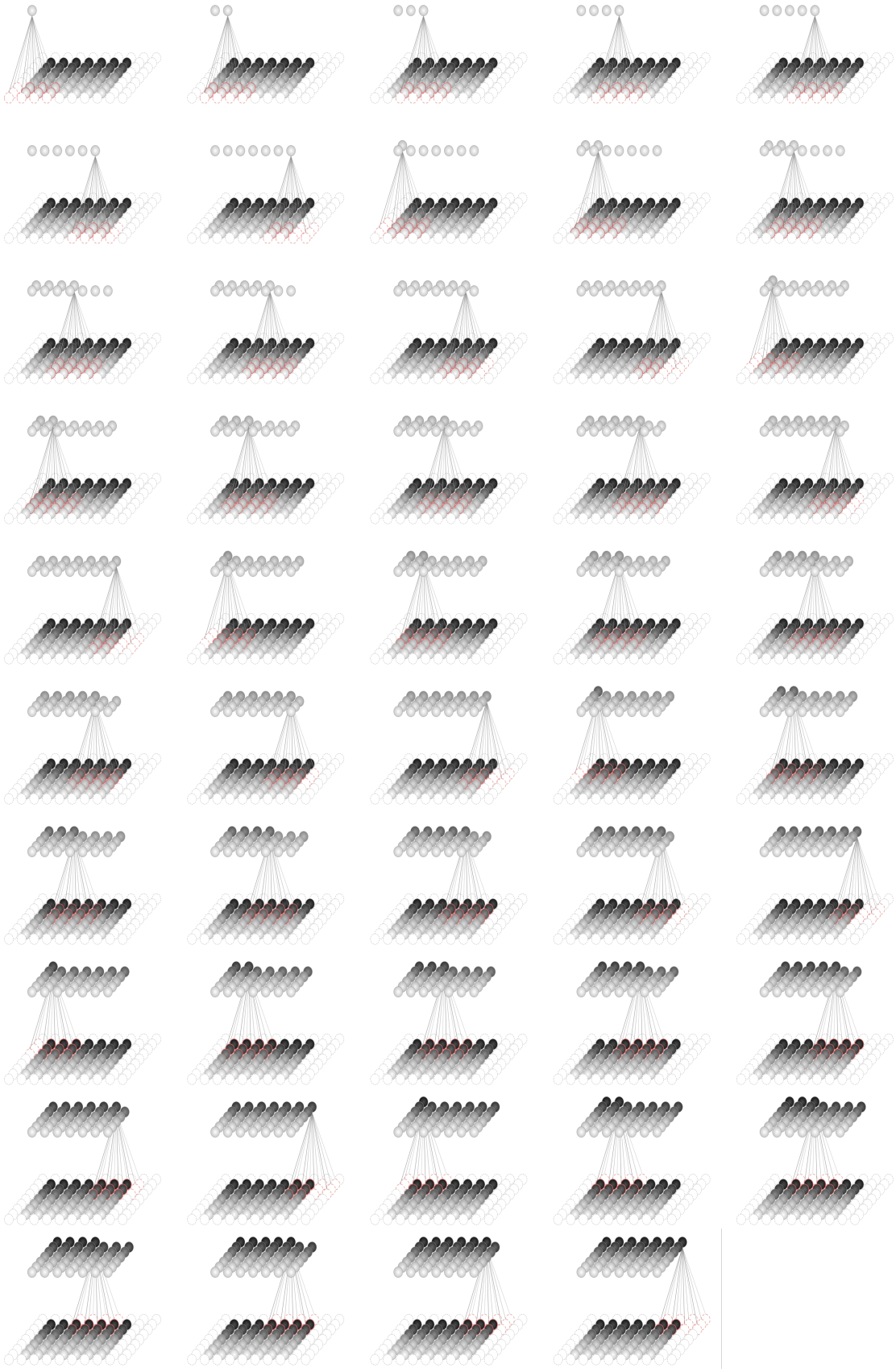


Figure 8.7. A kernel $\mathbf{k} \in \mathbb{R}^{3\times4\times1\times1}$ operating on a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times7\times1}$ with "SAME" padding.



Figure 8.8. A kernel $\mathbf{k} \in \mathbb{R}^{3\times3\times1\times1}$ operating on a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times7\times1}$ with "SAME" padding and stride of $(2, 2)$.

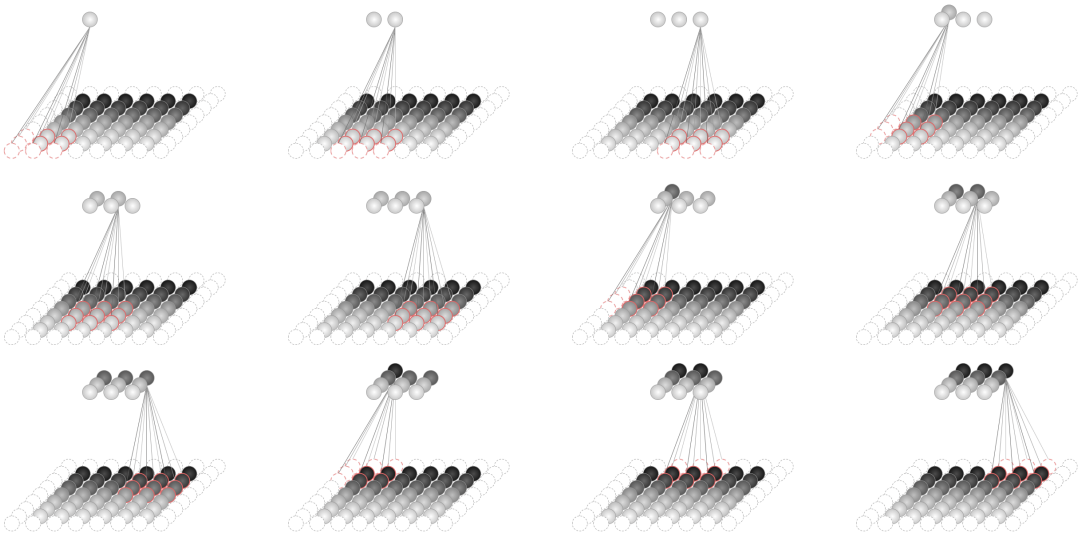


Figure 8.9. A kernel $\mathbf{k} \in \mathbb{R}^{3\times3\times1\times1}$ operating on a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times6\times1}$ with "SAME" padding and stride of $(2, 2)$

In all cases, for a spatial dimension of size i with padding p and a kernel of width k with stride s, the output size o is given by:

$$o = \left\lfloor \frac{i + p - k}{s} \right\rfloor + 1$$

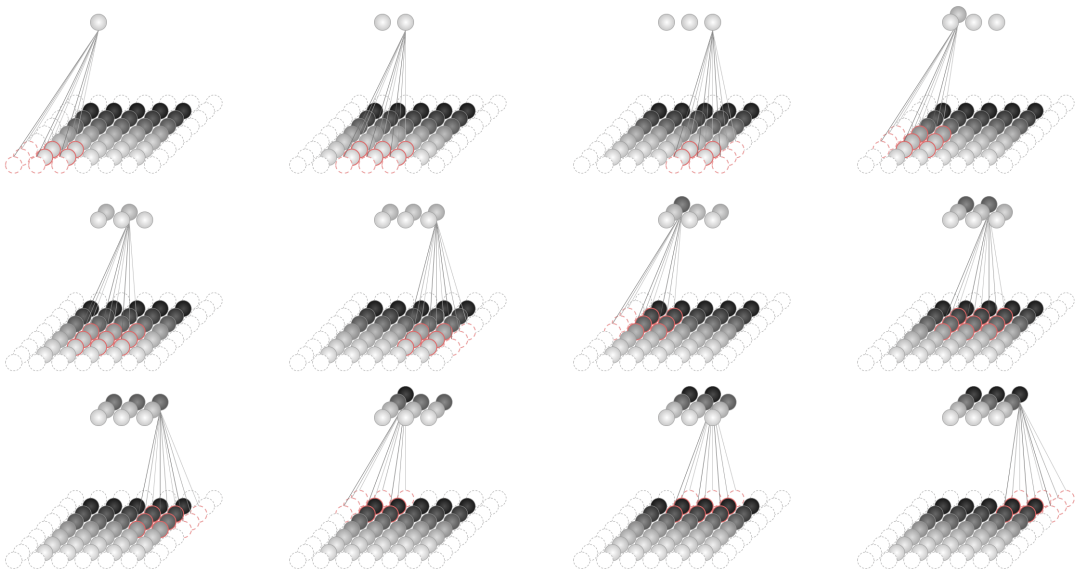


Figure 8.10. A kernel $\mathbf{k} \in \mathbb{R}^{3\times3\times1\times1}$ operating on a grayscale input image $\mathbf{x} \in \mathbb{R}^{7\times5\times1}$ with "SAME" padding and stride of $(2, 2)$.

References

[1] Dumoulin, V. & Visin, F. (2016), "A guide to convolution arithmetic for deep learning", arXiv preprint, abs/1603.07285.

[2] Fukushima, K. (1980), "Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position", *Biological Cybernetics*, **3** 6 (4), 193–202.

[3] Hubel, D. H. & Wiesel, T. N. (1959), "Receptive fields of single neurones in the cat's striate cortex", *Journal of Physiology* **148** (3), 574–91.

[4] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989) "Backpropagation Applied to Handwritten Zip Code Recognition", AT&T Bell Laboratories.

[5] Lu, Z., Pu, H., Wang, F. Hu, Z., & Wang, L. (2017) "The Expressive Power of Neural Networks: A View from the Width", Advances in Neural Information Processing Systems 30. Curran Associates, Inc., 6231–6239.

[6] Zhou, Y. & Chellappa, R. (1988), "Computation of optical flow using a neural network", in *IEEE International Conference on Neural Networks*, IEEE, 71-78.